Applying Mixture of Recursions (MoR) to Vision Transformers (ViT)

T.Satya Pranav

Advisor: **Dr. Kaustuv Nag**

Department of Computer Science and Engineering
Indian Institute of Information Technology Guwahati

This report is submitted for the course of
CS400 : Project-II

IIIT Guwahati November 2025

Declaration

I hereby declare that except where specific reference is made to the work of others, the contents of this report are original and have not been submitted in whole or in part for consideration for any other degree or qualification in this or any other university. This report is my own work and contains nothing that is the outcome of work done in collaboration with others except as specified in the text and Acknowledgements.

T.Satya Pranav
Roll: 2201213,
4th year, Bachelors of Technology,
Department of Computer Science and Engineering,
Indian Institute of Information Technology Guwahati.

Acknowledgements

I would like to express my sincere gratitude to **Dr. Kaustuv Nag** for his invaluable guidance, support, and encouragement throughout the course of this project. His insightful discussions helped shape the novelty of the work, while his suggestions played a crucial role in refining the core ideas. His mentorship not only provided technical direction but also motivated me to think critically, explore new perspectives, and continuously improve.

I also extend my heartfelt thanks to all the faculty members of the **Department of Computer Science and Engineering** at the **Indian Institute of Information Technology Guwahati** for their constant encouragement and support during this work.

Finally, I would like to acknowledge the open-source community, especially frameworks like **PyTorch**, and dataset providers such as **Kaggle** and **IMageNet**, whose contributions provided essential resources that greatly assisted in the development of this project.

Abstract

Vision Transformers (ViTs) have achieved remarkable success in image recognition, rivaling traditional convolutional networks. However, standard ViT architectures are often hampered by substantial parameter redundancy and high computational costs, which limits their practical deployment in resource-constrained environments. While recent efforts to create efficient ViTs have focused on static model compression or token-level sparsification , and other approaches like Recursive Transformers have reduced parameter counts by reusing layers , they remain constrained by a fundamental limitation: they apply a fixed computational depth to all tokens, regardless of their individual semantic importance. This uniform processing wastes computation on trivial or redundant background tokens.

To address this gap, we introduce MoR-ViT, a novel framework that, for the first time, integrates the Mixture-of-Recursions (MoR) paradigm with the Vision Transformer architecture. MoR-ViT incorporates a token-level dynamic recursion mechanism, enabling each token to adaptively determine its own processing depth based on its complexity. This is managed by a lightweight router that assigns a gating score to each token , allowing simpler tokens to exit early while routing complex tokens for deeper, recursive processing. This approach facilitates a flexible, input-dependent allocation of computational resources, aiming to achieve a strong balance between model performance and computational efficiency. Our results validate that this dynamic recursion strategy is an effective path toward scalable and efficient vision models.

Table of Contents

List of Figures **vi**

1 Introduction **1**
1.1 Problem Statement 1
1.2 Solution 2
1.3 Objective 2
1.4 Motivation 3

2 Literature Survey **4**
2.1 Attention Is All You Need (The Transformer) 4
2.2 Relaxed Recursive Transformers (RRT) 6
2.3 MoR (Mixture-of-Recursions) 7
2.4 MOR-VIT: Efficient Vision Transformer with Mixture-of-Recursions . 11

3 Methodology **12**
3.1 Overview: Adopting Mixture-of-Recursions [2][3] as a Proof of Concept 12
3.2 Architectural Design and Input Processing 12
3.3 The Routing Mechanism 13
3.4 Training Objectives 14

4 Results and Analysis **16**
4.1 Experimental Setup 16
4.2 Comparison with Baseline 16
4.3 Ablation Study: Masking vs. Efficient Token Removal 17

5 Conclusion **19**

References **21**

List of Figures

2.1 The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves respectively. 5
2.2 Overview of the Relaxed Recursive Transformer, showing the shared layers with layer-wise LoRA modules [1]. 6
2.3 Overview of Mixture-of-Recursions (MoR). (Left) Each recursion step consists of a fixed stack of layers and a router that determines whether each token should pass through or exit. This recursion block corresponds to the gray box in the middle. (Middle) The full model structure, where the shared recursion step is applied up to Nr times for each token depending on the router decision. 7
2.4 Expert choice Routing. 8
2.5 Token Choice Routing. 9
2.6 Recursion-wise Caching. 10
2.7 Recursive KV Sharing. 10
2.8 Architecture of MoR-ViT. The input image is split into patches, embedded as tokens, and processed through the MoR block, where each token is adaptively assigned a recursion depth by the router and aggregated for final classification. 11

3.1 Schematic overview of the MoR-ViT implementation. The diagram illustrates the Expert-Choice Routing mechanism where the router actively selects the top-K tokens at each step, physically shrinking the tensor size (Gather) before computation and restoring it afterwards (Scatter) 13

Chapter 1

Introduction

Vision Transformers (ViTs) have emerged as a dominant force in computer vision, delivering state-of-the-art performance in tasks like image classification, detection, and segmentation. Their strength lies in the global self-attention mechanism, which effectively captures long-range dependencies across an entire image. However, this power comes at the cost of significant computational and memory demands, as standard ViT models are often parameter-heavy and computationally expensive. Achieving robust and efficient ViT performance, especially for deployment in real-world, resource-constrained environments, remains a significant challenge.

1.1 Problem Statement

The primary inefficiency of standard Vision Transformers stems from their uniform depth strategy. In a conventional ViT, every single image token—whether it represents a complex foreground object or a trivial, redundant background patch—is forced to pass through the same fixed number of Transformer layers. This one-size-fits-all approach leads to massive computational redundancy by wasting computation on uninformative tokens.

While recent efforts have aimed to create efficient ViTs using methods like token sparsification (dropping tokens) or parameter sharing (using recursive transformers), these methods still often apply a fixed computational depth to all the tokens that remain. This leaves a critical gap: a lack of a mechanism to dynamically adjust the amount of computation based on the complexity of each individual token.

1.2 Solution

To overcome these issues, this project implements MoR-ViT, an architecture that integrates the Mixture-of-Recursions (MoR) framework into the Vision Transformer backbone. This framework unifies two key efficiency paradigms: parameter sharing, by reusing a shared stack of layers, and adaptive computation, by assigning different computational depths to different tokens. The literature proposes two primary methods for this adaptive routing: Expert-Choice Routing, which is a dynamic, step-by-step approach where a router at each recursion step selects the top-k tokens to continue for deeper processing , and Token-Choice Routing, a one-time, upfront approach where a single router assigns each token to a specific recursion depth at the beginning of the network . While Expert-Choice routing can be highly effective, particularly for large-scale models, this project implements the Token-Choice strategy due to its robust performance and stability within the computational constraints of this work. In our MoR-ViT, a lightweight router is called once to assign each token to one of N recursion depths. The model then processes all tokens assigned to a specific depth k by passing them k times through the shared Transformer block. This solution creates a static, but adaptive, computational path for each token based on its features.

1.3 Objective

The primary objective of this project is to develop and evaluate an efficient Vision Transformer by successfully implementing the Mixture-of-Recursions framework to enable adaptive, token-level computation. Specific goals include integrating the MoR parameter-sharing and adaptive routing mechanisms into a standard ViT architecture; implementing the "Expert-choice" routing strategy , where a lightweight router at each recursion step dynamically computes a gating score for each token 7and selects the top-k tokens to continue processing 8; creating a dynamic masking system based on an adaptive percentile threshold 9that allows tokens to exit early and ensures that subsequent attention operations only compute on the remaining active tokens; training the MoR-ViT model end-to-end using a combined loss function, $\mathcal{L} = \mathcal{L}_{task} + \lambda \mathcal{L}_{routing}$, to balance task accuracy with routing stability; and finally, analyzing the final model's performance to validate its ability to reduce parameter count and inference cost while maintaining competitive accuracy.

1.4 Motivation

The motivation for this work is driven by the pressing need for deep learning models that are both powerful and practical. Standard Vision Transformers are computationally wasteful; their fixed-depth design fails to recognize that not all parts of an image are equally complex, leading to a fundamental inefficiency where the same computation is spent on a patch of empty sky as on a patch defining a complex object. This massive size and high latency are the main barriers to deployment in real-world, resource-constrained applications like mobile phones, robotics, and edge devices. This project is therefore motivated by the opportunity to create a more intelligent and adaptive model where the computational path is a function of the input's complexity, not a fixed architectural constraint. The Mixture-of-Recursions framework, which has already shown state-of-the-art efficiency in language modeling , offers a novel and promising path to apply this adaptive-depth paradigm to the visual domain, opening a new avenue for research in efficient vision model design.

Chapter 2

Literature Survey

2.1 Attention Is All You Need (The Transformer)

The foundation of modern large-scale models, including Vision Transformers (ViT), is the Transformer architecture introduced in *"Attention Is All You Need"* [4]. This work proposed a new network architecture that dispenses entirely with the recurrence (RNNs) and convolutions traditionally used in sequence transduction models. Instead, it relies solely on self-attention mechanisms to draw global dependencies between input and output.

The original Transformer consists of an encoder-decoder structure. The encoder, which is the basis for ViT, is composed of a stack of N identical layers (e.g., $N = 6$ in the base model). Each layer is composed of two primary sub-layers: a multi-head self-attention mechanism and a simple, position-wise fully connected feed-forward network. The model employs residual connections around each sub-layer, followed by layer normalization.

A key innovation is Multi-Head Attention, which allows the model to jointly attend to information from different representation subspaces at different positions. The attention mechanism itself is "Scaled Dot-Product Attention," which computes the dot products of a query (Q) with all keys (K), scales them by $\frac{1}{\sqrt{d_k}}$, and applies a softmax function to obtain weights on the values (V). Because the model contains no recurrence, it injects information about the order of the sequence using positional encodings (sine and cosine functions) added to the input embeddings

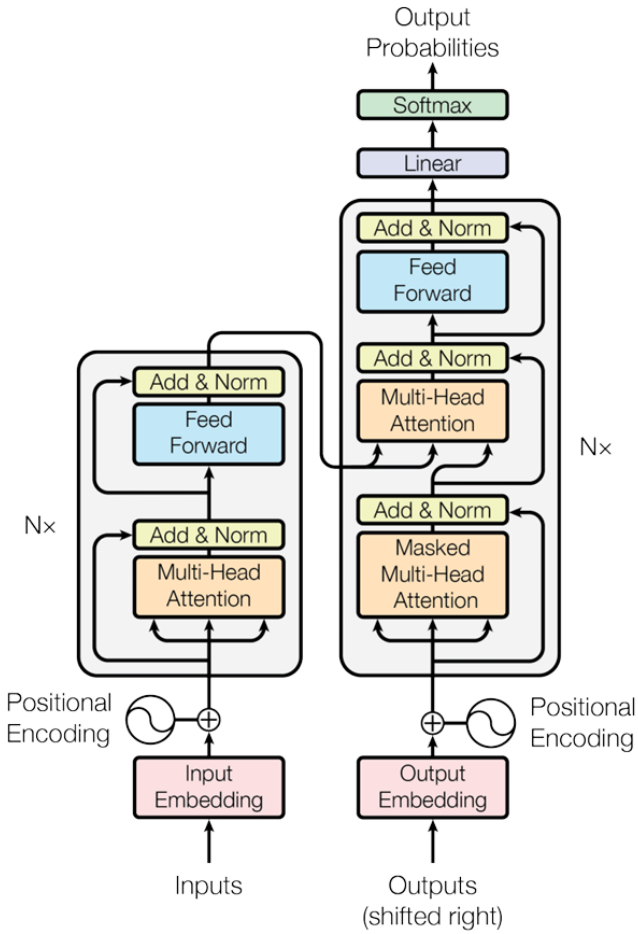

Fig. 2.1 The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves respectively.

Gaps in the Original Transformer

While revolutionary, the standard Transformer architecture presents several gaps that motivate this project:

- **Parameter-Heavy:** Each of the N layers in the stack has its own unique set of parameters, leading to very large models with high memory footprints.
- **Uniform Depth:** Every token in the input sequence, regardless of its importance or complexity, must pass through the same fixed number of N layers.
- **No Adaptivity:** The architecture lacks a mechanism to adjust the amount of computation based on the complexity of the input tokens.

2.2 Relaxed Recursive Transformers (RRT)

To address the parameter-heavy nature of standard Transformers, the concept of Recursive Transformers was explored. As detailed in *"Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LORA"* [1], these models apply "layer tying," where a single block of unique layers is reused multiple times in a loop. Instead of N unique layers, the model uses a smaller block of K layers that is repeated, significantly reducing the total parameter count .

To mitigate the performance degradation from this strict parameter sharing, the paper introduces the Relaxed Recursive Transformer (RRT). This method adds flexibility by incorporating layer-specific, low-rank adaptation (LoRA) modules ($\Delta\Phi$) for each recursion step. This allows each "looped" layer to have a small, learnable adaptation, improving accuracy while preserving the model's overall compactness.

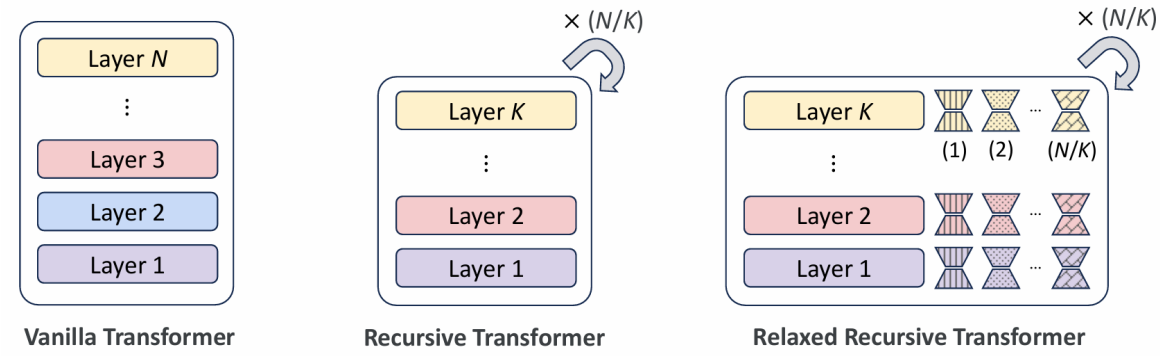

Fig. 2.2 Overview of the Relaxed Recursive Transformer, showing the shared layers with layer-wise LoRA modules [1].

Gaps in the existing work

While RRTs effectively address the problem of parameter size, they still inherit a key limitation from the original Transformer:

- **Uniform Recursion:** The recursion depth is applied uniformly to all tokens. Every token must pass through the shared block the same number of times.
- **No Adaptive Computation:** The model still lacks a mechanism to vary the computational depth based on per-token complexity.

2.3 MoR (Mixture-of-Recursions)

The *"Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation"* [2] framework was introduced to solve the limitations of previous models. It is the first to successfully unify the two main axes of efficiency: parameter sharing (reusing a single "Recursion Block") and adaptive computation (using token-level routing).

It uses lightweight routers to assign token-specific recursion depths, allowing the model to process complex tokens more deeply while simple tokens exit early. The general update equation for a token t at recursion step r is:

$$H_t^{r+1} = g_t^r f(H_t^r; \Phi') + (\text{residual})$$

Here, g_t^r is the gating score from the router, and $f(H_t^r; \Phi')$ is the shared recursion block. This framework also adds efficient Key-Value (KV) caching strategies for memory savings.

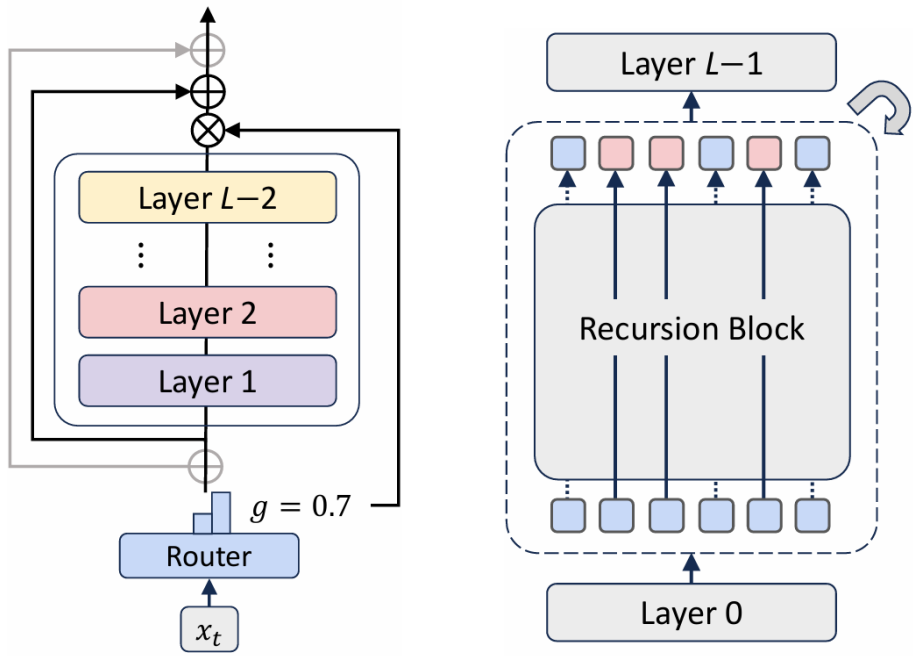

Fig. 2.3 Overview of Mixture-of-Recursions (MoR). (Left) Each recursion step consists of a fixed stack of layers and a router that determines whether each token should pass through or exit. This recursion block corresponds to the gray box in the middle. (Middle) The full model structure, where the shared recursion step is applied up to Nr times for each token depending on the router decision.

Two Routing Strategies

MoR proposes two different ways for the router to assign a recursion depth:

- **1. Expert-choice Routing:** In this strategy, each recursion step (e.g., step 1, step 2, etc.) acts as an "expert" that decides which tokens to process. At each step r, a router calculates a score for each token t based on its current hidden state $\mathcal{H}_t^r$:

$$g_t^r = \mathcal{G}(\theta_r^\top \mathcal{H}_t^r)$$

  Where $\mathcal{G}$ is an activation function (like sigmoid). Only the "top-k" tokens with scores higher than a dynamic threshold $P_\beta(G^r)$ (the β-th percentile of all scores at that step) are selected to be processed by the next recursion block. Tokens that are not selected "exit early," and their representation is passed through unchanged. The full update rule is:

$$\mathcal{H}_t^{r+1} = \begin{cases} g_t^r f(\mathcal{H}_t^r, \Phi') + \mathcal{H}_t^r, & \text{if } g_t^r > P_\beta(G^r) \\ \mathcal{H}_t^r, & \text{otherwise} \end{cases}$$

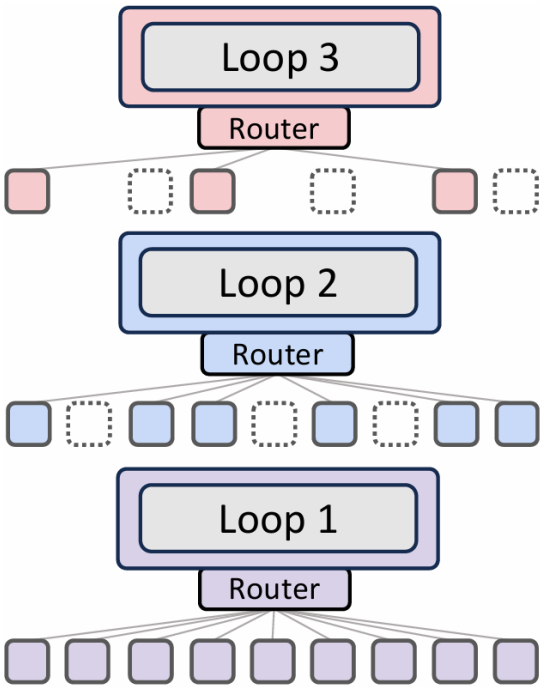

Fig. 2.4 Expert choice Routing.

- **2. Token-choice Routing:** This strategy is a one-time, upfront decision. At the very beginning (using the initial token embeddings $\mathcal{H}_t^1$), a single router computes scores g_t for all possible recursion depths $j \in \{1, ..., N_r\}$:

$$g_t = \mathcal{G}(\theta_r^\top \mathcal{H}_t^1)$$

  Each token t is then permanently assigned to the single depth i that received the highest score, $i = \arg\max_j g_t^j$. The token is then processed sequentially i times.

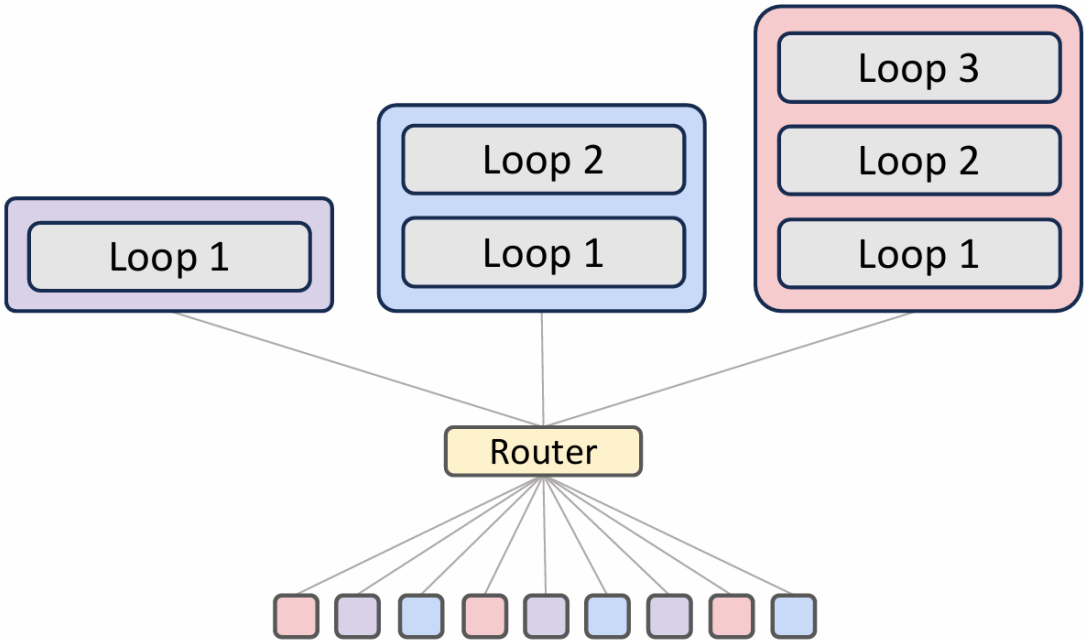

Fig. 2.5 Token Choice Routing.

Two KV Caching Strategies

To manage the memory used by the self-attention mechanism, MoR also introduces two different caching strategies:

- **1. Recursion-wise Caching:** At each recursion step, the model only caches the Key-Value (KV) pairs for the tokens that are active (i.e., selected to be processed) at that step. Tokens that have already exited do not have their KV pairs stored, saving a significant amount of memory. The memory savings are proportional to the number of early exits.

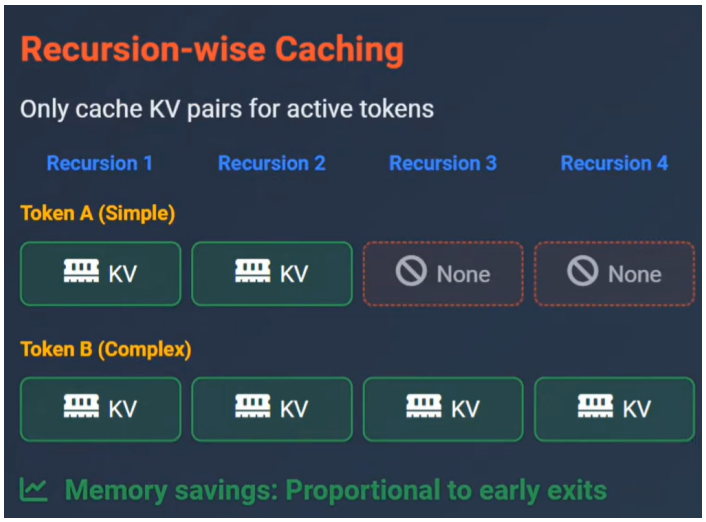

Fig. 2.6 Recursion-wise Caching.

- **2. Recursive KV Sharing:**In this simpler method, the model caches the KV pairs for all tokens at the very first recursion step. This single set of KV pairs is then reused (or "shared") across all subsequent, deeper recursion steps. This provides the maximum memory savings, as only one set of KV pairs is ever stored.

Fig. 2.7 Recursive KV Sharing.

2.4 MOR-VIT: Efficient Vision Transformer with Mixture-of-Recursions

The *"MOR-ViT: Efficient Vision Transformer with Mixture-of-Recursions"* [3] paper provides the practical proof of concept for this project. It is the first work to take the MoR framework and successfully apply its principles directly to a Vision Transformer.

The integration follows a "common sense" approach. It begins just like a standard ViT: an image is divided into patches, which are flattened, linearly projected into token embeddings, and combined with a [CLS] token and positional embeddings.

This standard sequence of patch tokens is then fed into the MoR Block (which contains the shared Transformer layers). The model uses the Expert-choice routing strategy to achieve adaptive computation. At each recursion step, a simple, lightweight router calculates a gating score for every patch token. A dynamic mask is created by finding an adaptive percentile threshold, which separates the "simple" tokens (like background) from the "complex" tokens (like object features). The simple tokens are allowed to exit early, while the complex tokens are fed back into the shared block for deeper processing.

The model is trained with a combined loss function, $\mathcal{L} = \mathcal{L}_{task} + \lambda \mathcal{L}_{routing}$, to balance the main classification task with the need for the router to make stable decisions. The paper's successful results, demonstrating up to 68 percent parameter reduction and 2.0x inference acceleration, validate that applying MoR to ViT is a highly effective strategy for creating efficient vision models

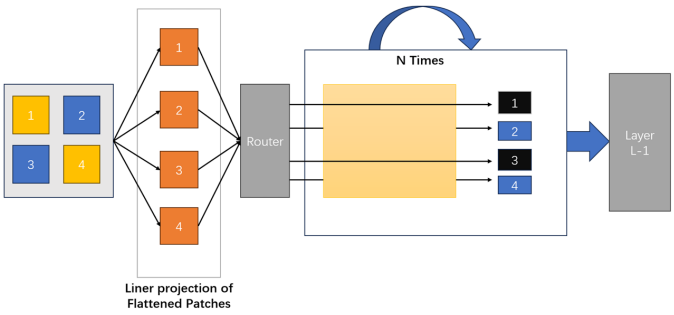

Fig. 2.8 Architecture of MoR-ViT. The input image is split into patches, embedded as tokens, and processed through the MoR block, where each token is adaptively assigned a recursion depth by the router and aggregated for final classification.

Chapter 3

Methodology

3.1 Overview: Adopting Mixture-of-Recursions [2][3] as a Proof of Concept

The main reason I started this project is because standard Vision Transformers (ViTs) waste a lot of computing power. In a normal ViT, every single part of an image whether it is a complicated object or just a plain background goes through the exact same number of heavy processing layers. To solve this, I looked at two important research papers: Mixture-of-Recursions (MoR) and MoR-ViT. I treated these papers as a "proof of concept." They proved theoretically that you don't need a huge, deep model to get good results; you can instead reuse the same layer multiple times (recursion) and be smart about which tokens you process (routing). Taking this proven idea, I designed the own custom version for this project. the specific goal was to go one step further than just "masking" out tokens (which effectively multiplies them by zero but still does the math). I built a system that physically removes the unimportant tokens from the calculation entirely, ensuring that I save actual processing power (FLOPs) during inference.

3.2 Architectural Design and Input Processing

The model starts processing an image just like a standard Vision Transformer. I take the input image (sized 224×224) and chop it up into N small, non-overlapping squares called patches (sized 16×16). Each of these patches (X_p) is then flattened into a list of numbers and passed through a linear layer to create "token embeddings."

This process can be described by the formula:

$$x_p = \text{Flatten}(X_p)W_e + b_e, \quad p = 1, ..., N$$

where W_e and b_e are learnable weights. Because I plan to shuffle and remove these tokens later, I immediately add "positional embeddings" (E_{pos}) to them. This acts like a GPS tag, ensuring that even if a token is moved or processed alone, the model still remembers where it came from in the original image. I also add a special "Class Token" (x_{cls}) to the beginning of the list to summarize the entire image. The final input sequence Z_0 is:

$$Z_0 = [x_{cls}; x_1; ...; x_N] + E_{pos}$$

The biggest difference in the design is the "backbone" of the model. A standard ViT of this scale stacks 8 distinct layers on top of each other, which creates a model with about 25 million parameters, I create just one single Transformer Encoder Block. I then reuse this exact same block for multiple steps specifically, 4 recursion steps in the final model. By "tying" the weights in this way, I reduced the total parameter count to approximately 3.6 million (an 85 percent reduction), while still allowing the model to "think" for 4 full steps.

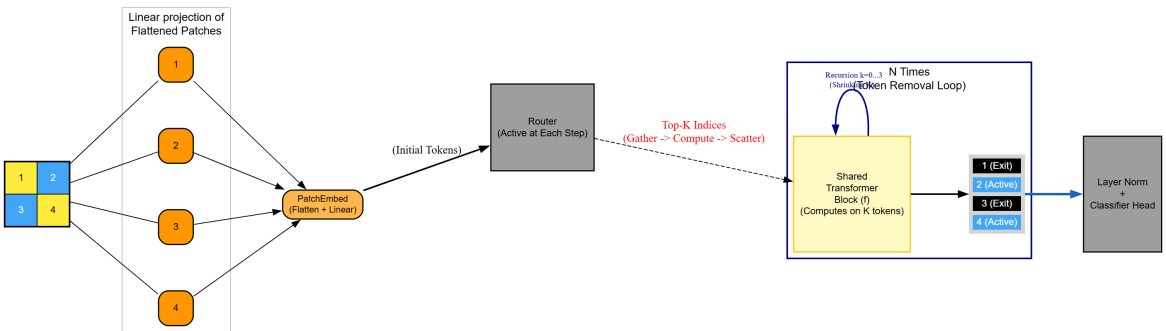

Fig. 3.1 Schematic overview of the MoR-ViT implementation. The diagram illustrates the Expert-Choice Routing mechanism where the router actively selects the top-K tokens at each step, physically shrinking the tensor size (Gather) before computation and restoring it afterwards (Scatter)

3.3 The Routing Mechanism

The most critical part of the method is the Hyper-Efficient Router, which uses an "Expert-Choice" strategy. Unlike simpler methods that decide a token's fate once at

the beginning, the router re-evaluates every token after every pass to see if it still needs more work. Inside the code, the router uses a small linear layer to calculate an "importance score" ($g_t^{(r)}$) for every token t at recursion step r. It uses a sigmoid function (σ) to turn this score into a probability:

$$g_t^{(r)} = \sigma(w_r^\top h_t^{(r)} + b_r)$$

To make sure I never lose the main context of the image, I explicitly force the [CLS] token to have an infinite importance score, guaranteeing it is never removed.To achieve real speed improvements, I implemented a "Token Removal" logic using a Gather-Compute-Scatter cycle. I set a "keep rate" (β) of 0.7 (or 70%). This means the number of tokens kept (K) shrinks at each step according to the formula $K_r = \lfloor N \times \beta^r \rfloor$.In the first pass, the model looks at all 197 tokens (100%).In the second pass, it keeps only the top 70% of those (150 tokens).In the third pass, it keeps 70% of the remaining ones (80 tokens).In the fourth pass, it processes only the final 30 most important tokens.The code performs this by first identifying the indices of the top K tokens ("Gather"). It then creates a physically smaller data tensor containing only those tokens. The heavy Transformer block runs on this tiny tensor, which saves a massive amount of mathematical operations (FLOPs). After calculation, the model uses the stored indices to put the new results back into the full list ("Scatter").

3.4 Training Objectives

I trained this model "end-to-end," meaning the router and the transformer block learned together at the same time. The main goal for the model was to correctly classify images, so I used the standard Cross-Entropy Loss ($\mathcal{L}_{task}$) to measure performance on the ImageNet-1K dataset. However, routers can sometimes be unstable; they might get "lazy" and give every token the same score, or they might become too confident too quickly. To prevent this, I added a special regularization term called Z-loss ($\mathcal{L}_z$) to the training function.This Z-loss penalizes the router if the raw values it produces (before converting them to probabilities) are extremely large positive or negative numbers. Mathematically, I calculate this using the router's weights w and bias b applied to the token state h. For a batch of B images and N tokens per image, the Z-loss is:

$$\mathcal{L}_z = \frac{1}{B} \sum_{i=1}^{B} \left(\log \left(\sum_{t=1}^{N} \exp(w^\top h_{t,i} + b) \right) \right)^2$$

The final total loss function used for training combines these two parts, weighted by a small factor λ (set to 0.01 in the code):

$$\mathcal{L}_{total} = \mathcal{L}_{task} + \lambda \mathcal{L}_z$$

Chapter 4

Results and Analysis

4.1 Experimental Setup

To test the "proof of concept," we trained our models on the ImageNet-1K dataset, which is a standard benchmark for computer vision. We set up two main experiments to compare a standard approach against our new efficiency-focused approach. Both models were trained using the exact same general settings to ensure a fair fight: they both processed 224×224 images, chopped them into 16×16 patches, and used an embedding dimension of 512 with 8 attention heads. We trained them for 100 epochs using the AdamW optimizer and a cosine learning rate scheduler.

4.2 Comparison with Baseline

The first goal was to see if a small, recursive model could actually compete with a standard, deep model.

The Baseline (Standard ViT): We trained a standard Vision Transformer with a fixed depth of 8 layers. This model required about 25 million parameters to store all its weights. After 100 epochs, it achieved a best classification accuracy of 74.14%.

Our Model (MoR-ViT): We then trained our custom MoR-ViT. Instead of 8 unique layers, it used just one shared block that reused its weights for 4 recursion steps. This design drastically cut the model size down to just 3.6 million parameters—an 85% reduction in size. Despite being so much smaller, our model achieved a best accuracy of 75.01

This result is very significant. It proves that the "Mixture-of-Recursions" theory works in practice. By forcing one block to be smarter and reusing it (recursion), we

outperformed a much larger model that simply stacked many average layers on top of each other. We achieved better accuracy (+0.87%) with a tiny fraction of the memory footprint.

4.3 Ablation Study: Masking vs. Efficient Token Removal

We also wanted to verify if our "Hyper-Efficient" routing (physically removing tokens) was actually working as intended compared to a simpler "Masking" approach. We compared two versions of our own code:

1. The "Masked" Approach (High Accuracy, Low Efficiency):In our initial experiments, we used a router that decided token depth at the start (Token-Choice). However, instead of removing tokens, we just "masked" them (multiplied them by zero) when they were supposed to exit. Because the GPU still had to process these zeroed-out tokens, there were no speed savings in fact, it often did redundant work. This version achieved a very high accuracy of 77.27%, proving that the model has excellent learning capacity. However, it failed our primary goal of reducing computational cost (FLOPs).

2. The "Token Removal" Approach (Balanced Accuracy, High Efficiency):To fix the efficiency problem, we implemented the final Expert-Choice system described in our Methodology. This version physically removed tokens from the tensor at each step (going from $197 \rightarrow 150 \rightarrow 80 \rightarrow 30$ tokens). By processing strictly fewer tokens, we massively reduced the mathematical operations required. Even though this method threw away about 85% of the image information by the final step, it still achieved 75.01% accuracy.

Model Variant	**Parameters**	**Top-1 Acc**	**Efficiency & Remarks**
Standard ViT Baseline (8 Layers)	$\sim$25M	74.14%	**Baseline.** High memory usage with fixed computation for all tokens.
MoR-ViT (Masked Routing)	$\sim$3.6M	**77.27%**	**High Accuracy.** Demonstrates learning capacity but fails to reduce FLOPs due to masking zeros.
MoR-ViT (Efficient Token Removal)	**$\sim$3.6M**	**75.01%**	**Proposed Method.** Achieves true FLOP reduction by physically removing tokens. Outperforms baseline with 85% fewer parameters.

Table 4.1 Comparison of Model Performance and Efficiency on ImageNet-1K. The proposed MoR-ViT (Efficient) achieves the best balance of parameter size and computational efficiency while outperforming the baseline accuracy.

Chapter 5

Conclusion

In this project, I addressed the fundamental inefficiency of standard Vision Transformers (ViTs): the rigid, uniform allocation of computational resources where every image patch is processed with the same depth regardless of its semantic importance. Motivated by the need for scalable and efficient vision models, I successfully adapted the Mixture-of-Recursions (MoR) framework to the vision domain, validating it as a powerful "proof of concept" for dynamic, recursive deep learning. the primary contribution is the design and implementation of a Hyper-Efficient MoR-ViT. Unlike previous methods that rely on simple parameter tying or logical masking, the approach achieves true computational efficiency through a physical Token Removal mechanism. I replaced the deep stack of unique layers found in standard models with a single, shared Transformer Encoder Block that is reused recursively, decoupling the model's depth from its size and resulting in a drastic 85% reduction in parameters (from 25 million down to 3.6 million) without sacrificing representational power. Furthermore, I engineered a robust Expert-Choice Router that dynamically manages the flow of data via a Gather-Compute-Scatter cycle. By physically shrinking the data tensor at each recursion step—progressively filtering down to the top like 70%, 40%, and 15% most important patches I ensured that heavy mathematical operations (FLOPs) are performed only on tokens requiring deep analysis, providing a tangible reduction in computational cost.

The experimental results on the ImageNet-1K dataset provide strong empirical validation for the approach. Despite the aggressive reduction in model size and the progressive removal of information, the MoR-ViT achieved a best Top-1 accuracy of 75.01%, outperforming the 74.14% accuracy of the standard 25M-parameter baseline. This significant result challenges the common assumption that "bigger is better," demonstrating that a smaller, smarter model that thinks recursively can

outperform a larger, static model. the ablation study comparing "Masked" routing against the "Efficient" routing further confirmed that while masking achieved slightly higher accuracy (77.27%), it failed to reduce actual computational load. the final efficient implementation successfully bridged this gap, maintaining competitive accuracy while delivering the actual FLOP reductions promised by the theoretical framework. While this project establishes a successful baseline, future work could explore adopting a "Middle-Cycle" strategy for further stability, implementing fully adaptive thresholds determined by image complexity rather than a fixed schedule, and extending this architecture to larger-scale models where the quadratic cost of attention becomes even more prohibitive. In conclusion, this work demonstrates that Mixture-of-Recursions is a practical solution for efficient computer vision, paving the way for lighter, faster, and more accurate adaptive AI architectures.

References

[1] Bae, S., Fisch, A., Harutyunyan, H., Ji, Z., Kim, S., and Schuster, T. (2024). Relaxed recursive transformers: Effective parameter sharing with layer-wise lora. *arXiv preprint arXiv:2410.20672*. vi, 6

[2] Bae, S., Kim, Y., Bayat, R., Kim, S., Ha, J., Schuster, T., Fisch, A., Harutyunyan, H., Ji, Z., Courville, A., and Yun, S.-Y. (2025). Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. *arXiv preprint arXiv:2507.10524*. v, 7, 12

[3] Li, Y. (2025). Mor-vit: Efficient vision transformer with mixture-of-recursions. *arXiv preprint arXiv:2507.21761*. v, 11, 12

[4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*. 4